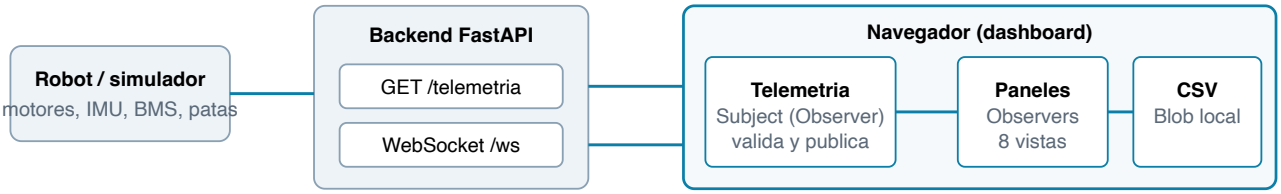
# Documento técnico · Dashboard de telemetría Unitree Go2

TP05 · Desarrollo de Aplicaciones II · UADE · Frontend web (HTML, CSS y JavaScript) · Repositorio: github.com/feliforte14/TP-Telemetria

## 1. Arquitectura

El sistema es una aplicación **cliente-servidor** con dos partes independientes que solo se conocen por la API. El **backend** (FastAPI, provisto por la cátedra) lee el robot o el simulador y expone `GET /telemetria` y `WS /ws`. El **frontend** (este trabajo) corre íntegramente en el navegador, sin framework ni servidor propio; Chart.js se carga por CDN. El dashboard es de solo lectura: no envía comandos al robot.




Dentro del navegador, `dashboard.js` se organiza en cinco bloques: (1) configuración y lógica pura (semáforo, buffer circular, validación, armado del CSV), (2) `Telemetria`, la capa de acceso a datos, (3) los paneles, (4) captura y exportación y (5) la aplicación que los conecta. La lógica pura no toca la pantalla, por lo que se prueba con Node (34 pruebas automáticas).

## 2. Decisiones de diseño

| Decisión | Motivo |
|---|---|
| **Patrón Observer** entre `Telemetria` y los paneles | `Telemetria` emite los eventos `dato` y `estado`; cada panel se suscribe con `en(evento, fn)`. La pantalla no sabe si el dato vino por WebSocket o polling, se puede cambiar la fuente sin tocar los paneles y un suscriptor que falla no corta a los demás (`try/catch` por aviso). |
| **Buffer circular de 300 muestras** | 30 s de historia a 10 Hz con memoria acotada: al llenarse se descarta la más antigua. Los gráficos no se recrean: se actualiza el mismo componente Chart.js sin animación, para no trabar el navegador (el mínimo exigido son 60 muestras). |
| **Semáforo de motores** (verde <40 °C, amarillo 40–60, rojo >60) | Función pura probada en los bordes (60,0 °C es amarillo). Se muestra con color *y* texto («Normal», «Precaución», «Sobrecalentado») para no depender solo del color. Las alertas saltan al pasar 60 °C y se rearman bajo 58 °C (histéresis), y varios motores simultáneos generan una sola muestra. |
| **CSV generado en el navegador** | Se arma con la API `Blob` y se descarga sin pasar por el servidor. Lleva BOM (Excel lee bien las tildes), una columna por magnitud (74 en el Go2) y escapa textos que empiezan con `=`, `+`, `-` o `@` para evitar inyección de fórmulas. |
| **Pérdida de conexión visible** | Indicador de estado, aviso rojo, paneles atenuados con los últimos valores y reintento automático. Un vigilante revisa cada segundo que lleguen datos (2 s sin datos = caído), porque el navegador no siempre avisa del corte. |
| **Seguridad y robustez** | Todo dato del servidor se escribe con `textContent` (nunca como HTML). Cada mensaje se valida contra el contrato; los inválidos se descartan y se cuentan. Cada pedido tiene tiempo máximo de 2 s. |
| **Interfaz** | Cuatro paneles obligatorios más *gemelo digital* (silueta cuyas patas se mueven con los ángulos reales y cuyas articulaciones se colorean por temperatura), tendencia por motor, temas oscuro y claro, y adaptación a tablet y celular. |

## 3. Polling vs. WebSocket

**Polling**: el cliente pregunta cada cierto tiempo (`GET /telemetria` cada 250 ms; la consigna admite hasta 500 ms). Es simple y funciona con cualquier proxy, pero cada dato cuesta un pedido HTTP completo, el dato puede tener hasta un intervalo de antigüedad y el servidor trabaja aunque no haya novedades. Se agenda el siguiente pedido recién cuando termina el anterior, para que nunca se acumulen. **WebSocket**: se abre una conexión persistente y el servidor

*empuja* cada mensaje apenas lo tiene (~10 por segundo); no hay encabezados repetidos ni espera, y los datos llegan con la cadencia real del robot.

| Criterio | Polling (250 ms) | WebSocket |
|---|---|---|
| Mensajes por segundo (medido) | ~4 | ~9,8 |
| Tráfico recibido (medido) | ~5,5 KB/s | ~13,4 KB/s |
| Latencia del pedido (medida) | ~7 ms por pedido, más hasta 250 ms de espera | no aplica (push) |
| Frescura del dato | hasta un intervalo de retraso | inmediata |
| Complejidad / compatibilidad | baja, universal, admite parámetros ( `?modelo=` ) | media: hay que reconectar; no admite parámetros |

**Decisión:** modo **automático** por defecto. Se intenta el WebSocket (datos más frescos y fluidos); si falla, se continúa por polling mientras se reintenta el WebSocket con espera creciente (1 a 10 s), y se vuelve a él apenas se recupera. El polling queda como respaldo y como único camino cuando se elige un robot ( `?modelo=` ). El usuario puede forzar cualquiera de los dos desde el selector «Fuente de datos». Con polling a 250 ms se cumple holgadamente el máximo de 500 ms de la consigna.

### Limitaciones conocidas

El simulador de la cátedra deriva torque, temperatura, roll y pitch del movimiento, por lo que con el robot real los valores serán otros. El servidor atiende un solo robot por vez y tiene CORS abierto: debe usarse solo en la red cerrada del laboratorio.